BASES DE DATOS II

FloatMate

Guia de preguntas y respuestas para la sustentacion

PostgreSQL	Bases distribuidas	PL/pgSQL

Proyecto: Terminal Norte + Terminal Sur
Preparado para exposicion oral y repaso tecnico

Como usar esta guia

Esta guia convierte la implementacion de FloatMate en respuestas breves y defendibles. No es necesario memorizar cada palabra: aprende la idea central, identifica el objeto SQL que la demuestra y responde con honestidad sobre la simulacion local.

Respuesta central: FloatMate aplica fragmentacion horizontal por terminal, federacion con postgres_fdw y vistas globales para ofrecer transparencia sobre los datos de Norte y Sur.

Punto de honestidad: en la practica academica los dos nodos son bases separadas dentro de una misma instalacion local. La arquitectura puede trasladarse a dos computadores cambiando el host y configurando la red.

Contenido

1. Arquitectura distribuida - 8 preguntas
2. Esquemas - 4 preguntas
3. Fragmentacion horizontal - 7 preguntas
4. Federacion y FDW - 8 preguntas
5. Vista global - 4 preguntas
6. Modelo relacional e integridad - 7 preguntas
7. Funciones almacenadas - 9 preguntas
8. Procedimientos almacenados - 5 preguntas
9. Triggers - 6 preguntas
10. Auditoria - 3 preguntas
11. Vistas materializadas y cursores - 4 preguntas
12. Preguntas dificiles y cierre - 5 preguntas

1. Arquitectura distribuida

1. ¿Que es FloatMate?

FloatMate es un sistema para administrar una flota de buses. Maneja vehiculos, conductores, rutas, viajes, controles de salida, combustible, mantenimientos, incidentes y usuarios.

2. ¿Donde esta la base de datos distribuida?

La informacion se divide entre floatmate_norte y floatmate_sur. Norte funciona como nodo coordinador y consulta Sur mediante postgres_fdw.

3. ¿Realmente son dos servidores diferentes?

En la demostracion son dos bases independientes dentro de una misma instalacion local de PostgreSQL. Es una simulacion academica de dos nodos. La arquitectura puede separarse fisicamente cambiando 127.0.0.1 por la IP de Sur y configurando la red.

4. ¿Por que utilizaron dos bases y no una sola?

Para representar dos terminales que administran sus datos localmente. Cada terminal conserva su fragmento, pero Norte puede presentar una consulta consolidada de toda la empresa.

5. ¿Cual es la base local y cual es la remota?

Desde Norte, floatmate_norte es local y floatmate_sur es remota. Aunque ambas esten en el mismo computador, FDW abre una conexion PostgreSQL independiente.

6. ¿Que es una base de datos distribuida?

Es una base cuyos datos se encuentran repartidos entre varios nodos, pero se ofrecen al usuario como un conjunto logico integrado.

7. ¿Cual es el nodo coordinador?

Terminal Norte. Alli se encuentran las tablas fisicas de Norte, las tablas extranjeras de Sur, las vistas globales y los objetos programables.

8. ¿Que pasa si Terminal Sur deja de responder?

Los datos locales de Norte siguen existiendo, pero las consultas que usan tablas remotas o vistas globales pueden fallar. La solucion usa federacion, no replicacion, por lo que Norte no conserva una copia permanente de Sur.

2. Esquemas

9. ¿Que es un esquema en PostgreSQL?

Es un espacio logico que organiza tablas, vistas, funciones, procedimientos y otros objetos. Se puede comparar con una carpeta dentro de la base de datos.

10. ¿Que esquemas utilizaron?

En Norte: floatmate para tablas fisicas, floatmate_remoto para tablas extranjeras, floatmate_global para vistas distribuidas y floatmate_programacion para funciones, procedimientos, triggers y auditoria. En Sur se usa floatmate.

11. ¿Por que no utilizaron public?

Porque public es el esquema predeterminado. Los esquemas propios mejoran la organizacion, los permisos y la separacion de responsabilidades, y corrigen la observacion realizada por el profesor.

12. ¿Hay algun objeto del proyecto en public?

No hay tablas, funciones ni procedimientos del negocio en public. Los objetos creados para FloatMate estan en esquemas propios.

3. Fragmentacion horizontal

13. ¿Que tipo de fragmentacion utilizaron?

Fragmentacion horizontal por terminal. Las tablas tienen la misma estructura, pero las filas se distribuyen segun terminal_codigo.

14. ¿Cual es el criterio de fragmentacion?

Norte acepta terminal_codigo = 'NOR' y Sur acepta terminal_codigo = 'SUR'.

15. ¿Como garantizan que un registro de Sur no se guarde en Norte?

Mediante restricciones CHECK. PostgreSQL rechaza cualquier fila que no cumpla el codigo de terminal asignado al nodo.

16. ¿Por que es horizontal y no vertical?

Porque se reparten filas de las mismas tablas. En una fragmentacion vertical se repartirian columnas.

17. ¿Como reconstruyen la tabla completa?

Con una vista global que aplica UNION ALL entre la tabla local y la tabla extranjera de Sur.

18. ¿Por que UNION ALL y no UNION?

Porque los fragmentos son disjuntos y no deberian contener duplicados. UNION ALL evita el costo de eliminar duplicados y conserva todas las filas.

19. ¿Como evitan colisiones de identificadores?

Norte utiliza el rango 1001-1015 y Sur el rango 2001-2015. En una implementacion mayor tambien se podrian usar UUID o secuencias coordinadas.

4. Federacion y FDW

20. ¿Que significa FDW?

Foreign Data Wrapper. Es el mecanismo de PostgreSQL para consultar fuentes externas. Se usa postgres_fdw porque el origen remoto tambien es PostgreSQL.

21. ¿Que hace CREATE SERVER?

Registra en Norte la direccion, el puerto y el nombre de la base remota. El servidor extranjero se llama servidor_terminal_sur.

22. ¿Que es un USER MAPPING?

Relaciona un usuario local con las credenciales utilizadas para entrar al servidor remoto.

23. ¿Que permisos tiene floatmate_fdw?

Tiene CONNECT, USAGE y SELECT sobre Sur. Es una cuenta de solo lectura y aplica el principio de minimo privilegio.

24. ¿Que hace IMPORT FOREIGN SCHEMA?

Importa las definiciones de las tablas de Sur y las registra en Norte como tablas extranjeras dentro de floatmate_remoto.

25. ¿Que es una tabla extranjera?

Es un objeto local que representa una tabla almacenada en otra base. La consulta se envia a Sur en tiempo real; no es una copia de los datos.

26. ¿Como demuestra que la consulta es remota?

Con EXPLAIN VERBOSE sobre una tabla de floatmate_remoto. El plan debe mostrar un Foreign Scan.

27. ¿Federacion es lo mismo que replicacion?

No. La federacion consulta datos que siguen en otro nodo. La replicacion mantiene copias de los datos en varios nodos.

5. Vista global

28. ¿Que es la vista global?

Es una representacion logica que une Norte y Sur. El usuario consulta una sola vista sin conocer la ubicacion fisica de cada registro.

29. ¿Donde estan las vistas globales?

En el esquema floatmate_global. Hay vistas para las nueve entidades del modelo.

30. ¿Que hace v_operacion_viajes?

Relaciona viajes, vehiculos, conductores, rutas y controles de salida de ambas terminales.

31. ¿Una vista almacena fisicamente los datos?

Una vista normal no. Ejecuta su consulta cada vez. Una vista materializada si conserva el resultado hasta que se ejecuta REFRESH MATERIALIZED VIEW.

6. Modelo relacional e integridad

32. ¿Cuantas tablas tiene cada nodo?

Nueve: usuarios, conductores, vehiculos, rutas, viajes, controles_salida, abastecimientos, mantenimientos e incidentes.

33. ¿Cuantos registros hay?

Cada tabla tiene 15 registros en Norte y 15 en Sur. La vista global presenta 30 registros por entidad.

34. ¿Cual es la tabla central?

Viajes, porque relaciona un vehiculo, un conductor y una ruta, y se conecta con controles de salida, abastecimientos e incidentes.

35. ¿Que es una clave primaria?

Es el atributo que identifica de forma unica cada fila, por ejemplo viaje_id.

36. ¿Que es una clave foranea?

Es un atributo que referencia la clave primaria de otra tabla, por ejemplo viajes.vehiculo_id hacia vehiculos.vehiculo_id.

37. ¿Que tipos de integridad implementaron?

Integridad de entidad con claves primarias, referencial con claves foraneas, de dominio con CHECK, unicidad con UNIQUE, obligatoriedad con NOT NULL y reglas avanzadas con triggers.

38. ¿Hay claves foraneas entre Norte y Sur?

No directamente. Cada nodo protege sus relaciones locales. Las vistas globales consolidan informacion, pero no crean claves foraneas entre servidores.

7. Funciones almacenadas

39. ¿Que es una funcion almacenada?

Es codigo guardado en PostgreSQL. Recibe parametros, ejecuta una logica y devuelve un resultado reutilizable.

40. ¿Como se crea una funcion?

Se usa CREATE OR REPLACE FUNCTION, se declaran parametros, tipo de retorno, lenguaje y cuerpo. Las funciones del proyecto se crean con nombre de esquema.

41. ¿Donde crearon las funciones?

En floatmate_programacion, nunca en public.

42. ¿Que tipos de funciones implementaron?

Escalares, tabulares, SQL, PL/pgSQL, JSONB, cursores, sobrecarga y funciones de trigger.

43. ¿Que hace fn_kilometros_para_mantenimiento?

Recibe un vehiculo y devuelve cuantos kilometros le faltan para el proximo mantenimiento.

44. ¿Que significa RETURNS TABLE?

Que la funcion devuelve varias filas y columnas y puede consultarse en la clausula FROM como una tabla.

45. ¿Que es la sobrecarga de funciones?

Es usar el mismo nombre con tipos de entrada diferentes. fn_total_combustible(bigint) calcula por vehiculo y fn_total_combustible(varchar) calcula por terminal.

46. ¿Que significa STABLE?

Que la funcion puede consultar tablas, pero no debe modificarlas, y mantiene un resultado estable dentro de una sentencia.

47. ¿Cual es la diferencia entre IMMUTABLE, STABLE y VOLATILE?

IMMUTABLE siempre produce el mismo resultado para los mismos parametros; STABLE puede consultar la base sin modificarla; VOLATILE puede modificar datos o cambiar entre llamadas.

8. Procedimientos almacenados

48. ¿Que diferencia hay entre una funcion y un procedimiento?

La funcion devuelve un resultado y suele invocarse con SELECT. El procedimiento se ejecuta con CALL y esta orientado a realizar un proceso o modificar datos.

49. ¿Que procedimientos implementaron?

Cambiar estado de vehiculo, programar viaje, iniciar viaje, cerrar viaje y refrescar la vista materializada de consumo.

50. ¿Como se ejecuta un procedimiento?

Con CALL y el nombre calificado, por ejemplo CALL floatmate_programacion.sp_cambiar_estado_vehiculo(1001, 'MANTENIMIENTO').

51. ¿Que significa INOUT?

Es un parametro que recibe un valor y tambien puede devolver uno. sp_programar_viaje usa INOUT para devolver el viaje_id generado.

52. ¿Que pasa si se intenta programar un vehiculo ocupado?

El procedimiento valida disponibilidad y lanza una excepcion. La operacion queda cancelada.

9. Triggers

53. ¿Que es un trigger?

Es una accion automatica que PostgreSQL ejecuta ante un INSERT, UPDATE o DELETE.

54. ¿Que diferencia existe entre BEFORE y AFTER?

BEFORE ocurre antes del cambio y puede rechazarlo. AFTER ocurre despues y es apropiado para auditoria.

55. ¿Que valida tr_validar_viaje?

Que vehiculo, conductor y ruta existan, pertenezcan a la misma terminal y que los pasajeros no superen la capacidad.

56. ¿Que valida tr_validar_inicio_viaje?

Impide iniciar sin control aprobado, con vehiculo o conductor no disponibles, o con el viaje en un estado incorrecto.

57. ¿Que son NEW y OLD?

NEW es la fila nueva y OLD la fila anterior. UPDATE tiene ambas; INSERT usa NEW y DELETE usa OLD.

58. ¿Que es TG_OP?

Es la variable especial que informa si el evento fue INSERT, UPDATE o DELETE.

10. Auditoria

59. ¿Que informacion almacena la auditoria?

Fecha, usuario de base, operacion, tabla, clave del registro, datos anteriores, datos nuevos y direccion del cliente.

60. ¿Por que usan JSONB?

Porque permite guardar filas completas con estructuras diferentes sin crear una tabla de auditoria distinta para cada entidad.

61. ¿Cuando se activa la auditoria?

Despues de INSERT, UPDATE o DELETE sobre cualquiera de las nueve tablas locales de Norte.

11. Vistas materializadas y cursores

62. ¿Que diferencia hay entre vista y vista materializada?

La vista normal ejecuta su consulta cada vez. La materializada guarda el resultado y debe refrescarse cuando cambian los datos.

63. ¿Como actualizan la vista materializada?

Con CALL floatmate_programacion.sp_refrescar_consumo_vehiculo(), que ejecuta REFRESH MATERIALIZED VIEW.

64. ¿Que es un cursor?

Es un mecanismo para recorrer el resultado de una consulta fila por fila usando OPEN, FETCH, EXIT y CLOSE.

65. ¿Por que se prefieren operaciones por conjuntos?

Porque PostgreSQL esta optimizado para procesar conjuntos. Los cursores son utiles cuando se necesita procesamiento secuencial, pero suelen ser menos eficientes.

12. Preguntas dificiles y cierre

66. ¿Que limitacion tiene la implementacion local?

Ambos nodos viven en la misma instalacion fisica. Demuestra fragmentacion, federacion y transparencia, pero no ofrece independencia fisica ni tolerancia real a la caida de un computador.

67. ¿Por que sigue siendo util la simulacion?

Porque cada nodo es una base independiente y la comunicacion usa postgres_fdw. Para mover Sur a otro servidor basta cambiar el host y configurar red y autenticacion.

68. ¿La vista global garantiza integridad referencial?

No por si sola. Las claves foraneas protegen cada nodo; la vista global se usa para reconstruir y consultar los fragmentos.

69. ¿Que pasa si se repite un identificador entre nodos?

Podrian aparecer ambiguedades. Por eso se usan rangos separados; en produccion se podrian usar UUID o secuencias coordinadas.

70. ¿Como comprobaron que todo funciona?

Con 03_validacion_total.sql y 02_pruebas_y_ejemplos.sql. Estos revisan cantidades, fragmentacion, FDW, relaciones, vistas, funciones, procedimientos, triggers y auditoria.

Guion de cierre en 60 segundos

FloatMate administra la operacion de una flota y distribuye sus datos por terminal. Terminal Norte almacena el fragmento NOR y Terminal Sur el fragmento SUR. Las restricciones CHECK garantizan la fragmentacion horizontal. Desde Norte usamos postgres_fdw para importar las tablas de Sur dentro de floatmate_remoto, y las vistas de floatmate_global reconstruyen la informacion completa con UNION ALL. El modelo tiene claves primarias, foraneas, restricciones e indices. Adicionalmente, floatmate_programacion contiene funciones, procedimientos, triggers, auditoria y una vista materializada. Toda la entrega se comprueba con scripts automaticos de validacion.

Comandos que conviene recordar

Objetivo	Ejemplo
Comprobar acceso remoto	SELECT count(*) FROM floatmate_remoto.vehiculos;
Consultar la vista global	SELECT * FROM floatmate_global.v_operacion_viajes;
Demostrar Foreign Scan	EXPLAIN (VERBOSE, COSTS OFF) SELECT * FROM floatmate_remoto.viajes;
Ejecutar una funcion	SELECT floatmate_programacion.fn_kilometros_para_mantenimiento(1001);
Ejecutar un procedimiento	CALL floatmate_programacion.sp_cambiar_estado_vehiculo(1001, 'MANTENIMIENTO');
Refrescar vista materializada	CALL floatmate_programacion.sp_refrescar_consumo_vehiculo();

Referencias tecnicas

PostgreSQL - postgres_fdw: https://www.postgresql.org/docs/current/postgres-fdw.html

PostgreSQL - IMPORT FOREIGN SCHEMA: https://www.postgresql.org/docs/current/sql-importforeignschema.html

PostgreSQL - CREATE FUNCTION: https://www.postgresql.org/docs/current/sql-createfunction.html

PostgreSQL - CREATE TRIGGER: https://www.postgresql.org/docs/current/sql-createtrigger.html

PostgreSQL - Materialized Views: https://www.postgresql.org/docs/current/rules-materializedviews.html

> **Antes de presentar: ejecuta 03_validacion_total.sql y 02_pruebas_y_ejemplos.sql. Si ambos terminan correctamente, tendras evidencia tecnica para respaldar tus respuestas.**